LAB 2

DIGITAL Electronics 2B

EEDE316

Semester 1

Luther Xabisile Ndondo

T241657

Lab Overview:

Interfacing SPI OLED with STM32F401RE, featuring LDR light sensor, potentiometer (ADC→DAC feedback), and single PWM LED. OLED displays live graphs (light trends, pot waveform), ADC/DAC values, enabling signal processing visualization.

Components:

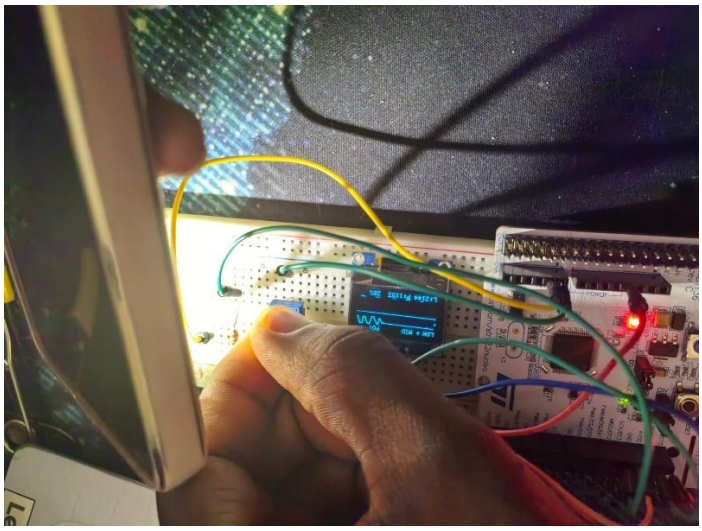
• STM32F401RE Nucleo-64

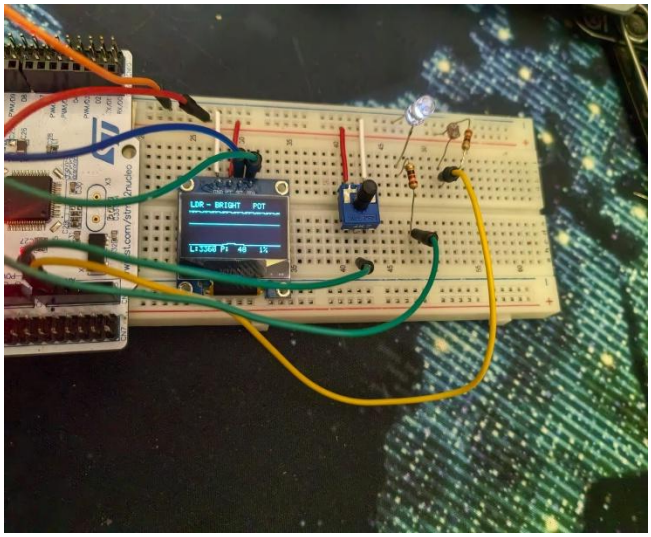
• SSD1306 SPI OLED (128x64)

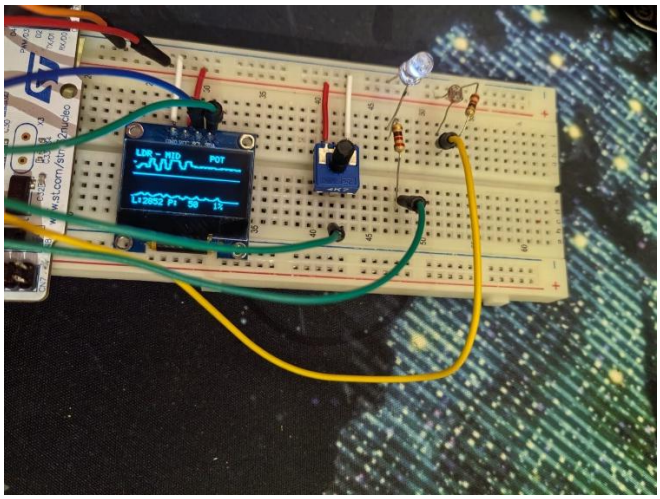
• LDR + 10kΩ (light sensor)

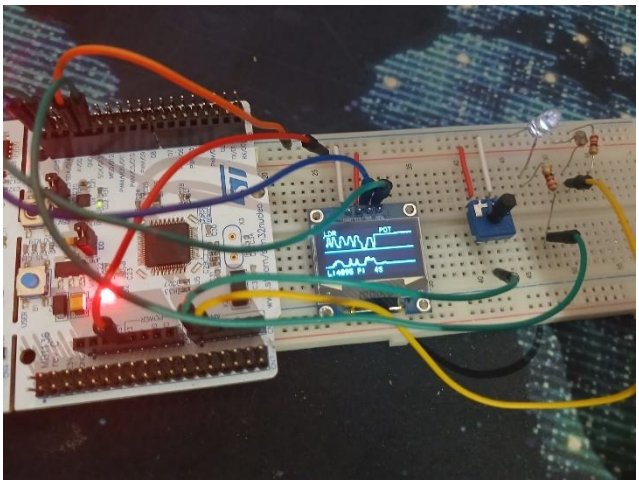
• 10kΩ Potentiometer (ADC input → DAC output)

• 1x LED + 220Ω (PWM) • Wires, breadboard









CODE :

```
#include "main.h"

#include <string.h>   /* memset, memcpy, memmove */

#include <stdio.h>    /* snprintf              */


/* — HAL peripheral handles (used by HAL driver functions) — */

ADC_HandleTypeDef  hadc1;   /* ADC1  — reads PA0 and PA1       */

I2C_HandleTypeDef  hi2c1;   /* I2C1  — talks to OLED on PB8/PB9 */

TIM_HandleTypeDef  htim2;   /* TIM2  — generates PWM on PA5    */


/* — Global sensor values, updated every loop iteration ——— */

uint32_t ldrValue = 0;

uint32_t potValue = 0;


/* — Forward declarations (functions defined below main) ——— */

void SystemClock_Config(void);

static void MX_GPIO_Init(void);

static void MX_ADC1_Init(void);

static void MX_I2C1_Init(void);

static void MX_TIM2_Init(void);
```

```
#define OLED_ADDR  (0x3C << 1)  /* I2C address 0x3C, shifted for HAL */
#define OLED_W     128          /* display width  in pixels        */
#define OLED_H     64           /* display height in pixels        */
#define OLED_PAGES 8            /* 8 pages of 8 rows each = 64 rows  */

static uint8_t fb[OLED_PAGES][OLED_W];

/* ── Send one command byte to the OLED ──────────────────────
  The SH1106 needs a "control byte" (0x00) before every
  command byte to tell it "this is a command, not data".  */
static void sh_cmd(uint8_t cmd)
{
   uint8_t buf[2] = {0x00, cmd};
   HAL_I2C_Master_Transmit(&hi2c1, OLED_ADDR, buf, 2, HAL_MAX_DELAY);
}



static void sh_flush(void)
{
   uint8_t tx[OLED_W + 1];
   tx[0] = 0x40;  /* control byte: "data follows" */

   for (uint8_t page = 0; page < OLED_PAGES; page++) {
      sh_cmd(0xB0 + page);  /* select page (row band)         */
      sh_cmd(0x02);       /* column start = 2 (SH1106 offset) */
      sh_cmd(0x10);       /* column high nibble = 0          */
      memcpy(&tx[1], fb[page], OLED_W);  /* copy 128 pixels    */
      HAL_I2C_Master_Transmit(&hi2c1, OLED_ADDR, tx, OLED_W + 1, HAL_MAX_DELAY);
   }
}
```

```
/* — Clear the framebuffer (all pixels OFF) ———————— */
static void fb_clear(void)
{
   memset(fb, 0, sizeof(fb));
}

/* — Set or clear one pixel at (x, y) ————————————————
  x = column (0=left, 127=right)
  y = row   (0=top,  63=bottom)
  The bit position within a page byte = y % 8          */
static void fb_pixel(uint8_t x, uint8_t y, uint8_t on)
{
   if (x >= OLED_W || y >= OLED_H) return;  /* bounds check */
   if (on)
      fb[y / 8][x] |=  (1u << (y % 8));   /* set bit   */
   else
      fb[y / 8][x] &= ~(1u << (y % 8));   /* clear bit */
}

/* — Draw a vertical line from y0 to y1 at column x ———— */
static void fb_vline(uint8_t x, uint8_t y0, uint8_t y1)
{
   if (y0 > y1) { uint8_t t = y0; y0 = y1; y1 = t; } /* swap if needed */
   for (uint8_t y = y0; y <= y1; y++) fb_pixel(x, y, 1);
}

/* — Draw a horizontal line at row y from x0 to x1 ———— */
static void fb_hline(uint8_t x0, uint8_t x1, uint8_t y)
{
   for (uint8_t x = x0; x <= x1; x++) fb_pixel(x, y, 1);
}
```

MAIN CODE

```
============================================================
 * SECTION 4 — MAIN PROGRAM
 * ========================================================== */
int main(void)
{
  /* --- Initialise all hardware --- */
  HAL_Init();            /* HAL library + systick timer  */
  SystemClock_Config();    /* 84 MHz system clock        */
  MX_GPIO_Init();        /* user button on PC13        */
  MX_ADC1_Init();        /* ADC1 for PA0 and PA1       */
  MX_I2C1_Init();        /* I2C1 on PB8/PB9 for OLED   */
  MX_TIM2_Init();        /* TIM2 PWM on PA5 for LED    */

  sh_init();             /* power on and blank the OLED  */
  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);  /* start PWM  */

  /* --- Splash screen for 2 seconds --- */
  fb_clear();
  fb_str( 4, 10, "STM32 DASHBOARD");
  fb_str(10, 26, "LDR + POT + PWM");
  fb_str(22, 42, "SIGNAL MONITOR");
  sh_flush();
  HAL_Delay(2000);

  /* --- Main loop: read → process → display → repeat --- */
  while (1)
  {
    /* STEP 1: Read both sensors */
    ldrValue = adc_read(ADC_CHANNEL_0);   /* LDR on PA0 */
    potValue = adc_read(ADC_CHANNEL_1);   /* POT on PA1 */
```

```
/* STEP 2: Set LED brightness from pot
   Maps 0–4095 ADC range onto 0–1000 PWM timer range */
uint32_t duty = potValue * 1000UL / 4095UL;
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, duty);

/* STEP 3: Push new readings into scrolling history */
push_sample();

/* STEP 4: Calculate derived display values */
int8_t  trend   = calc_trend();
uint8_t pwm_pct = (uint8_t)(potValue * 100UL / 4095UL);

/* STEP 5: Build the frame ———————————————— */
fb_clear();

/* --- Top label row (y=0) --- */
fb_str(0, 0, "LDR");   /* left label for LDR graph  */

/* Trend arrow immediately after "LDR" text */
if     (trend ==  1) fb_arrow_up(22, 0);  /* ↑ rising  */
else if (trend == -1) fb_arrow_dn(22, 0);  /* ↓ falling */
else                  fb_dash(22, 0);      /* — stable  */

/* Light level word: depends on raw ADC value */
if     (ldrValue > 3000) fb_str(34, 0, "BRIGHT");
else if (ldrValue > 1500) fb_str(34, 0, "MID");
else                      fb_str(34, 0, "DIM");

fb_str(88, 0, "POT");  /* right label for POT graph */

/* --- LDR waveform (y 8–27) --- */
draw_wave(ldr_hist, GRAPH_BOT_LDR);

/* --- Divider line between the two graphs --- */
fb_hline(0, 127, 28);
```

```
    /* --- POT waveform (y 29–55) --- */
    draw_wave(pot_hist, GRAPH_BOT_POT);

    /* --- Bottom strip: live values (y=57) ---
      Format:  L:3241 P:2180  73%
      L = LDR raw value, P = POT raw value, % = PWM duty */
    char buf[32];
    snprintf(buf, sizeof(buf), "L:%4lu P:%4lu %3u%%",
          ldrValue, potValue, pwm_pct);
    fb_str(0, 57, buf);

    /* STEP 6: Send completed frame to OLED (~50ms per frame) */
    sh_flush();
    HAL_Delay(50);
  }
}
```